Report for assignment 4

Project

Name: Android

URL: https://github.com/CatimaLoyalty/Android

Description: Catima is an Android App that stores the store loyalty and membership cards on your device, instead of having to carry them around.

Onboarding experience

We chose the same project as the last assignment, so the onboarding is the same as last time. Additionally, we tried to run the Android Studio this time to visualize the whole process and our changes by interacting with the UI. The Android Studio started up a mirror phone to simulate the app running on an Android phone.

Overview of issue(s) and work done.

Title: FEATURE REQUEST: Choose Default Currency

URL: https://github.com/CatimaLoyalty/Android/issues/2886

Summary: When adding a gift card to the app, the default currency type is now POINTS. So, the feature is required to let the user set their default currency.

Work done: We added an entry point in the Setting page for the user to choose their own default currency. The entry point is a dropdown menu, showing all the currency type options. And every time the card is being created, it will show the currency type that was set as default.

Scope (functionality and code affected):
LoyaltyCardEditActivity::onCreate(): Handle showing the correct currency type (set default / previously chosen type)
SettingsActivity.kt: Handle the dropdown menu options and selection logic on the settings page.
settings.xml, strings.xml, preference.xml: setting page UI

Requirements for the new feature or requirements affected by functionality being refactored

- **R0**
  - Title: Preferred currency setting is available in Settings.
  - Description: The application provides a setting for “Preferred currency” in the Settings screen so that the user can choose which currency new cards should default to.
  - Acceptance criteria:
    - The Settings screen contains a “Preferred currency” option.
    - The “Preferred currency” setting is placed under the Card creation category in Settings.
    - The user can change the “Preferred currency” setting by selecting exactly one option from a list of available currencies.
    - The currently active “Preferred currency” selection is displayed in the UI.
- **R1**
  - Title: Default preferred currency is Points.
  - Description: If the user has not previously selected a preferred currency, the application defaults the preferred currency setting to “Points”.
  - Acceptance criteria:
    - The “Preferred currency” setting contains an option for “Points”
    - On the first run (or when no other preference has been set), the preferred currency is set to “Points”.
    - The “Points” label is presented using the current Locale’s translation.

- **R2**
  - Title: Preferred currency selection is persisted.
  - Description: When the user changes the preferred currency, the application persists the selection and restores it on subsequent application launches.
  - Acceptance criteria:
    - After the user selects a preferred currency option, that option remains selected after the app is closed and then reopened.
- **R3**
  - Title: New cards use the preferred currency.
  - Description: When the user creates a new card, the application uses the currently selected preferred currency setting when initialising the card’s currency/balance type.
  - Acceptance criteria:
    - If the user has a preferred currency set, a newly created card uses that option as its default currency/balance type.

Requirement	Verified by Test(s)
R0	Verified implicitly by other tests and manually through the emulator.
R1	verifyDefaultCurrencyPreferenceInitiallySetToPoints()
R2	LoyaltyCardEditActivityDefaultCurrencyTest::manualCurrencyPersistsAfterSave()
R3	onCreate_setsBalanceTypeFromSettings_SEK() onCreate_setsBalanceTypeWhenSettingsNull_points()

UML class diagram and its description

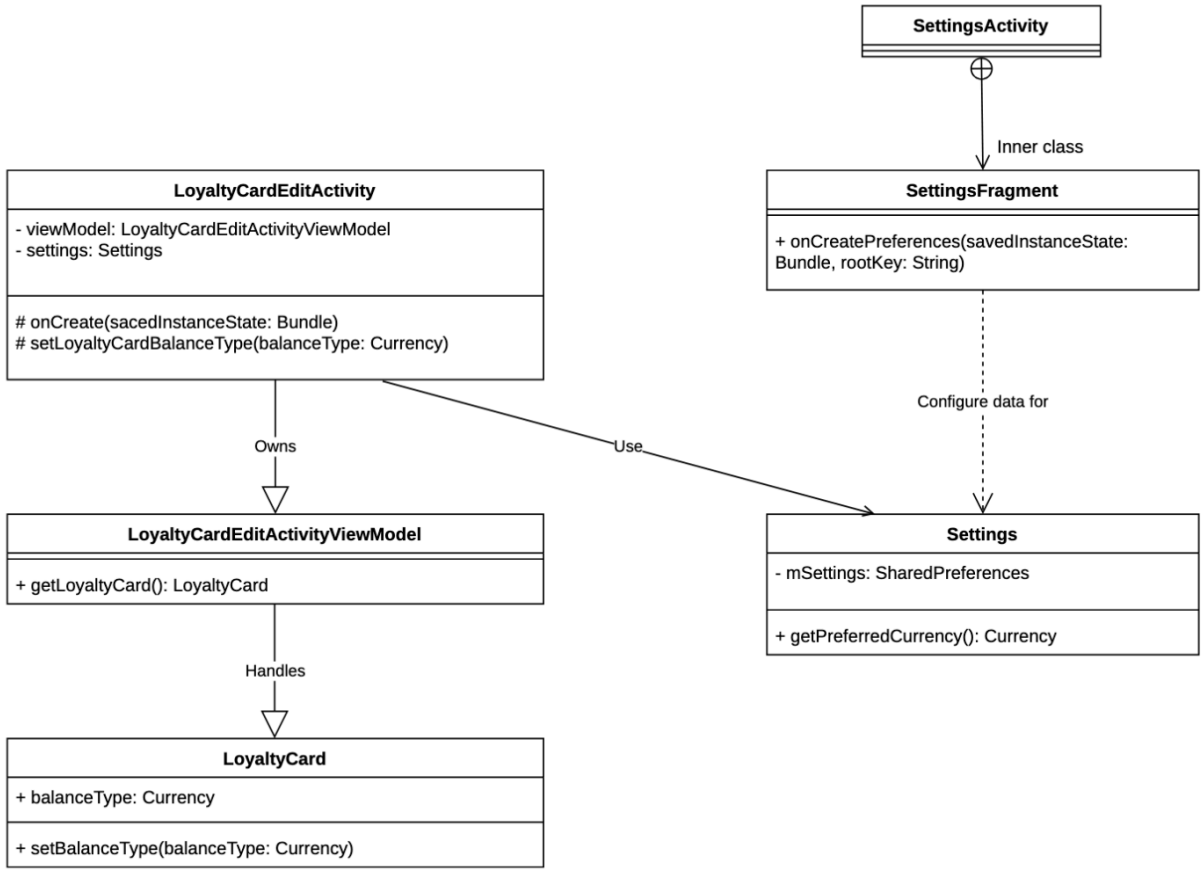

Figure 1: UML Class Diagram

- + (Public): The method or field is accessible by any other class in the project.
- # (Protected): The method or field is accessible only within its own class, by subclasses or other classes in the same package.
- - (Private): The method or field can only be accessed from within the class where it is defined.

- **LoyaltyCardEditActivity:** This class serves as the controller or UI Entry Point. When it is created (onCreate), it handles the initialization of the default currency by interfacing with the Settings class.
- **LoyaltyCardEditActivityViewModel:** This class acts as the state holder, managing the data independently of the UI lifecycle. The class provides the link between the activity and the underlying LoyaltyCard model.
- **LoyaltyCard:** The entity representing a card. This class contains the balanceType field (of type Currency) where the card's specific currency is persisted.
- **SettingsActivity & SettingsFragment:** These represent the configuration UI. They provide the interface for the user to modify the ListPreference, which defines the preferred default currency for all new cards.
- **Settings:** This is a type of data access class that encapsulates the logic for retrieving preferences in the application. It includes the getPreferredCurrency() method to fetch the user's default selection from SharedPreferences.

- Owns (Association)

LoyaltyCardEditActivity -> ViewModel:
The Activity is responsible for creating and holding the ViewModel instance during its lifecycle.

- Use (Association)

LoyaltyCardEditActivity -> Settings:
The Activity calls methods in the Settings class to fetch the user's preferred default currency

- Handles (Association)

ViewModel -> LoyaltyCard:
The ViewModel acts as a mediator that manages the LoyaltyCard object and its data (like the balanceType)

- Configure data for (Dependency)

SettingsFragment -> Settings:
The fragment provides the UI where the user saves the currency setting, which Settings later reads from the app's memory.

Code changes

Patch

git diff sync/upstream-main..issue-2886-add-default-currency

Key changes/classes affected

Catima uses a traditional multi-Activity Android architecture, where each screen is represented by a dedicated Activity class. These activity classes implement presentation logic, navigation handling, and lifecycle management, and constitute the UI/presentation layer. Data access is handled through helper classes such as DBHelper. Additionally, there are various domain models, such as LoyaltyCard and Group. This architectural style (multi-Activity) is considered a traditional approach, while the modern approach is to use a single-Activity/fragment-based architecture [1].

Our modifications were confined to the presentation layer, specifically within SettingsActivity and LoyaltyCardEditActivity. Additionally, some smaller additions were made to preferences.xml, strings.xml, and Settings.java files to support configuration handling in SettingsActivity. Furthermore, tests were added for the implemented functionality, however, these are not relevant to the architecture. No changes were made to the data layer or domain model structures, ensuring architectural consistency and minimal cross-layer impact.

A key architectural consideration involved the choice of where in the Android activity lifecycle to place the initialization of a loyalty card's currency/balance type. The preferred currency is supposed to act as a default applied only when a card is first created, while subsequent edits should use the card's persisted currency. For this reason, the currency is set to the preferred option in onCreate(), behind a guard that ensures its only called when initializing a new card. By having this logic in onCreate(), we avoid re-applying the defaults on repeated lifecycle transitions such as onResume(), which was the solution of a different PR that was never completed.

Overall experience

Selina:

- Through this experience, I learned how working on a large-scale project differs significantly from completing individual assignments. In an open-source context, I had to consider not only functionality, but also consistency with the existing architecture, avoiding regressions, and ensuring that current tests and features were not broken. I also had to quickly understand a complex codebase within a limited time.

  This experience made me realize the importance of clean and well-structured project architecture, which enables others to read, maintain, and contribute effectively. Additionally, I recognized how crucial it is to write a clear and well-defined issue,

including a proper problem description and acceptance criteria, so that external contributors can understand the context and onboard efficiently.

Rasmus:

- One key takeaway from this assignment (and last one) is the importance of clear documentation, both in the code and in READMEs. Well-written READMEs simplify the onboarding experience by a lot, and lower the threshold for getting started in a project you're not familiar with. Furthermore, documenting the code helps with understanding both the purpose of methods and how they interact with each other. In the Catima project, the documentation is close to nonexistent, and this made it take longer for us to get started and to understand the project.
- Using the way-of-working template was also new to me, and I would say that it helped improve our team's performance. It forced us to continually reflect on and assess our collaboration and identify areas to improve by answering questions such as, "Does everyone feel comfortable with the project?" and "Do we have the necessary tools to perform our tasks?". By comparing the way-of-working in the first assignment to this one, it is clear that we improved as a team.

Emma:

- I learned the importance of well-documented code to make others be able to understand what the code does. Our project had no java-doc which made it harder to understand what individual functions do.
- I also learned how accessible it is to contribute to an open-source project. Even though getting familiar with the codebase can be hard, once you understand it well enough, the process of picking up an issue and making a meaningful contribution is surprisingly straightforward.
- Using the Essence standard to evaluate our teamwork was very good. It made us think about how we worked and how it could be improved. I think we really improved our teamwork during the course, which is reflected in the difference of the way-of-working checklist of assignment 1 and assignment 4.

Filippa:

- Through this project I have gotten a greater sense on what it is like to contribute to an open source project. My main take away is probably the importance of requirements and having a clear goal. The issue we picked was pretty loosely defined as it was a feature request, leading to a lot of back-and-forths about how broad our scope really was, and in the end we took a pretty conservative approach, not expanding much on the original design. Especially with it being an open-source project, it felt important not to disturb the original structure even though it had parts that were lacking, e.g. documentation and test coverage, leading to a sort of balancing act that led one to approach the problem in a different way than when you have full control over a project. I also learnt a lot about testing and got a greater sense of what parts of logic needs to be tested or not, as I had to work on testing code I had not written myself.

- By evaluating ourselves against the Essence standard, it has been easier to get a “big-picture view” of where we are at in the greater context of software engineering as a practice, making it easier to see how the concepts we are learning scale and relate to work being done in the real world. It also worked well as a framework to drive discussions of what we needed to improve on as a team. In particular, the Essence standard focuses a lot on tools and practices used and necessitates that we constantly adapt and evolve our usage, which probably made us less prone to just sticking with the first-best approach and not questioning it further. One such example was that we implemented GitHub templates after the first assignment, after some feedback that it was hard to make sure that all our commits and issues were consistent.

Jacob:

- My main takeaway from this project was the challenge of figuring out and verifying code against a large code base. I learned more about programming in java and how an android project is structured. Working on the integration tests I gained new knowledge on the android specific testing environment, particularly how to use ActivityScenario to programmatically launch and control an Activity’s lifecycle.
- This project also gave me a sense of what it is like to contribute to an open source codebase which was really interesting. It feels like a lot more thought went into respecting the design patterns and existing structure compared to a solo project, which made the development a large bit slower in the beginning.
- As this is the fourth assignment we have completed together, it feels like we have grown as a team. I think we have an easier time making decisions faster now, and I feel like there is also more trust when we divide the work amongst ourselves. This is probably because tools and practices are working well and we can focus on the task at hand. I would also say we have communicated really well throughout and our team is in the “working well” state.

Evaluation of SEMAT

- Benefits

  Opportunity: Allowing users to choose a default currency instead of assuming “Points” reduces repetitive manual configuration when creating new cards. Since many gift cards use real currencies, this better aligns the system with actual user needs.

  Stakeholder : Users who primarily use non-point currencies benefit from fewer repeated actions, leading to reduced mistakes and less frustration. This improves overall user satisfaction and perceived product quality.

  Software System: The default currency logic is executed in onCreate() rather than formatBalanceField() to avoid overwriting user-selected values. This preserves

separation of concerns, prevents unintended side effects, and maintains consistency with the project's coding structure.

- Drawbacks
  In terms of Opportunity, the issue was quite minor and an isolated request from a single user rather than a clear demand from the project owner or other Stakeholder. This led to us having to make assumptions about the Requirements of the issue, and as we are not entirely familiar with the Software System, more time was generally spent in the Solution area rather than the Endeavour part. In the end, we made a very simple implementation that might need to be expanded upon later, possibly making for unnecessary extra work in the future compared with if there would have been a clearer issue in the first place. In other words, the balance between the different areas might have been lacking due to our approach.

- Limitations
  While the project proceeded relatively smoothly, several limitations can be identified. Regarding the **Stakeholders** alpha, the maintainers were slow to respond when we requested permission to work on the issue. Due to time constraints, we chose to proceed without explicit approval, which introduced a risk of wasted effort if the maintainers would have declined or redirected our contribution. Concerning **Requirements**, the issue lacked clear acceptance criteria, forcing us to define our own interpretation of what a successful solution would look like. Although we were able to draw on feedback from a previously declined pull request, this is an uncertain foundation, as it does not guarantee alignment with the maintainers' current expectations. Finally, with respect to the **Software System** alpha, our limited familiarity with the codebase meant that our understanding of the system was necessarily shallow. While we addressed the issue, we cannot be fully confident that our changes did not introduce unintended side effects elsewhere in the system, which is an inherent limitation when contributing to an unfamiliar open source project.

Effort spent

Emma

1. plenary discussions/meetings: 8.5h
2. discussions within parts of the group:
3. reading documentation: 2h
4. configuration and setup: 2h
5. analyzing code/output: 4h
6. writing documentation:
7. writing code: 3h

8. running code: 1h

Total: 20.5h

Filippa

1. plenary discussions/meetings: 8.5h
2. discussions within parts of the group:
3. reading documentation: 3h
4. configuration and setup: 1.5h
5. analyzing code/output: 3h
6. writing documentation: 1h
7. writing code: 2h
8. running code: 1h

Total: 20h

Jacob

1. plenary discussions/meetings: 6.5h
2. discussions within parts of the group: -
3. reading documentation: 5h
4. configuration and setup: 2h
5. analyzing code/output: 2h
6. writing documentation: -
7. writing code: 4h
8. running code: 1h

Total: 20.5h

Rasmus

1. plenary discussions/meetings: 8.5h
2. discussions within parts of the group: -
3. reading documentation: 3.5h
4. configuration and setup: 1.5h
5. analyzing code/output: 2h
6. writing documentation: 2h
7. writing code: 3h
8. running code: 1h

Total time: 21.5h

Selina

1. plenary discussions/meetings: 8.5h
2. discussions within parts of the group: -
3. reading documentation:
4. configuration and setup: 2h
5. analyzing code/output: 1h
6. writing documentation: 1h
7. writing code: 7.5h
8. running code: 0.5 h

Total: 20.5h

References

[1] *Guide to app architecture*, Android Developers, Nov. 24, 2025. Accessed: Mar. 1, 2026. [Online]. Available: https://developer.android.com/topic/architecture.